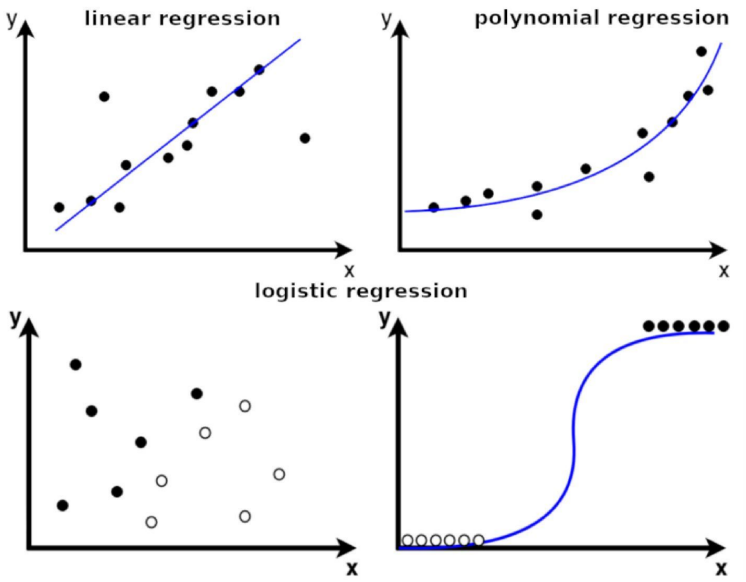
Evaluating Model Performance

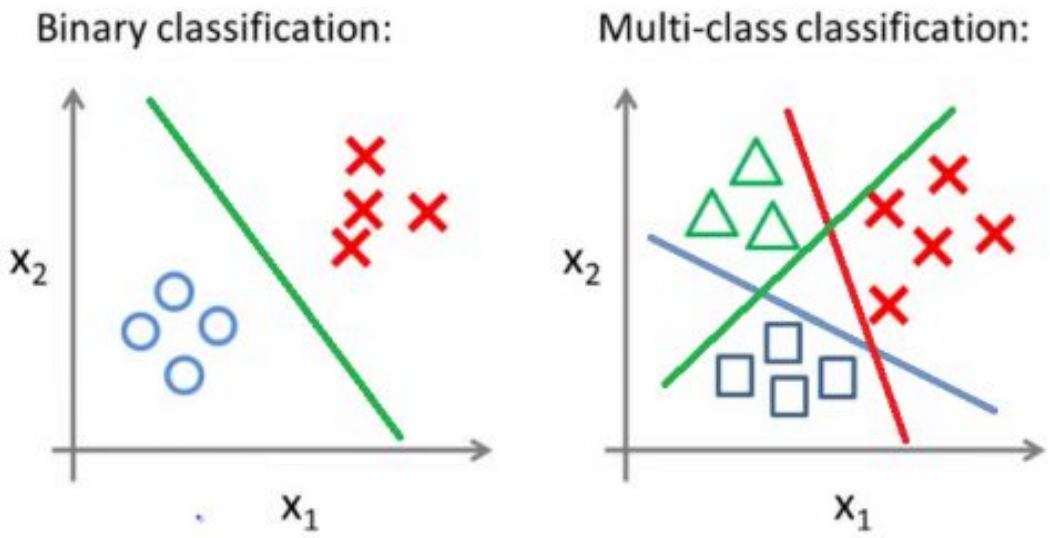
DATA 201 -- WEEK 9A

SPRING 2026

Topics

- Statistical Learning vs Machine Learning (inference vs prediction)
- Training and Fitting ML Models
  - Train/test split
  - Bias balance tradeoff
- Cross-validation





Regression & Classification: Statistical and Machine Learning

Complementary Methods, Goals, Focus, Approach

STATISTICAL LEARNING

- Understand relationships between predictors and target: *Why is this happening?*
- Make inferences about data, assuming mathematical models and probabilities
- Small, structured datasets meeting assumptions for data distributions
  - Normal error, homoscedasticity, independence
- More interpretable
- Choose to understand relationship between variables, explain a process, or when data is limited

MACHINE LEARNING

- High-accuracy predictions*: What will happen next?*
- Pattern detection without assumptions regarding data
- Large, high-dimensional datasets with minimal assumptions
- More often “black boxes
- Choose for high accuracy prediction, automated pattern recognition, handling massive complex datasets

Feature	Statistical Learning	Machine Learning
Main Objective	Inference and Explanation	Prediction & Accuracy
Key Metric	P-values, confidence intervals	MSE, RMSE, MAW
Model Complexity	Low-dimension, interpretable	High-dimension, less interpretable
Data Size	Structured, probability distribution	Unstructured
Optimization	Mathematical (loss function)	Iterative (e.g. gradient descent)

- GLMs have statistical learning focus on inference, parameter interpretation, and explaining relationships between predictors and target
- Linear regression is a bridge between statistical learning and machine learning

Example: Linear Regression Implementation in Python

Feature	Scikit-learn(Linear Regression)	statsmodels
Predictive modelling	✓	✗
Statistical inference	✗	✓
P-values and confidence intervals	✗	✓
Residual diagnostics	✗	✓
Speed on large datasets	✓	✗
Automatically includes intercept	✗ (manual adjustment needed)	✓

Linear Regression Example: $\beta_0 + \beta_1 X + \epsilon$

X: hours studied
Y: exam score

STATISTICAL LEARNING

- Main question: *Do hours studied have an effect on exam scores?*
- Fit model using all data
- Estimate coefficients (β_i)
- Compute: confidence intervals and p-values of coefficients; R^2 for model
- We want to know:
  - Does the predictor have an effect on the target?
  - How big is the effect?
- We need assumptions for linearity

MACHINE LEARNING

- Main question: *How well can we predict exam scores for students (generally) based on hours studied?*
- Train/test split
- Train on training data
- Cross-validation
  - Compare models
  - Tune hyperparameters
  - Estimate performance
- Evaluate on test data
- We want to know:
  - How accurate are predictions on new students?
  - Does the model generalize?

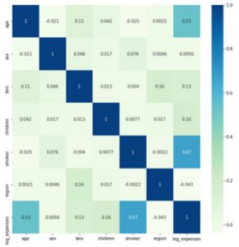
Same: mathematical model **Different: what we want to know, how data is used, how we evaluate success.**

LINEAR REGRESSION

1 EDA

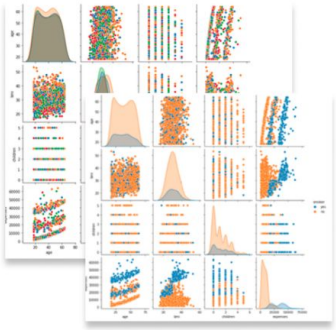
Heatmap

sns.heatmap(df.corr( ))



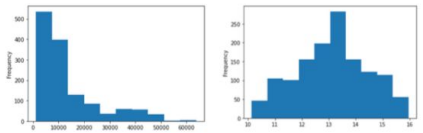
Scatterplot

sns.pairplot(df, hue = "...")



2 Feature Engineering

Log Transformation *np.log2( )*



One Hot Encoder

pd.get_dummies( )

smoker_no	smoker_yes	region_northeast	region_northwest	region_southea
0	1	0	0	
1	0	0	0	
1	0	0	0	
1	0	0	1	
1	0	0	1	

Label Encoder

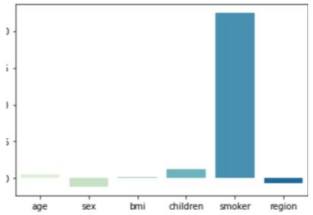
LabelEncoder( )

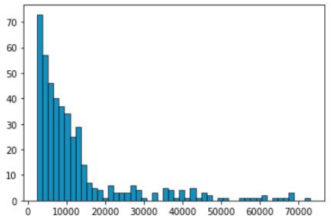
sex	bmi	children	smoker	region
0	27.9	0	1	3
1	33.8	1	0	2
1	33.0	3	0	2
1	22.7	0	0	1
1	28.9	0	0	1

3 Implementation

Feature Importance

sns.barplot(x = X_train.columns, y = coef)





Linear Regerssion

model = LinearRegression( )
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
*coef = model.coef_*
*intercept = model.intercept_*

4 Evaluation

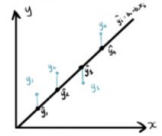
R Squared

coefficient of determination

$$R^2 = 1 - \frac{RSS}{TSS}$$

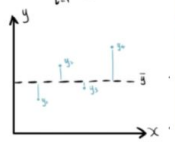
Residual Sum of Squares (RSS)

$$RSS = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$



Total Sum of Squares (TSS)

$$TSS = \sum_{i=1}^{n} (y_i - \bar{y})^2$$



Mean Absolute Error

$$MAE = \frac{1}{n} \sum_{i=1}^{n} |y_i - \hat{y}_i|$$

Mean Squared Error

$$MSE = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

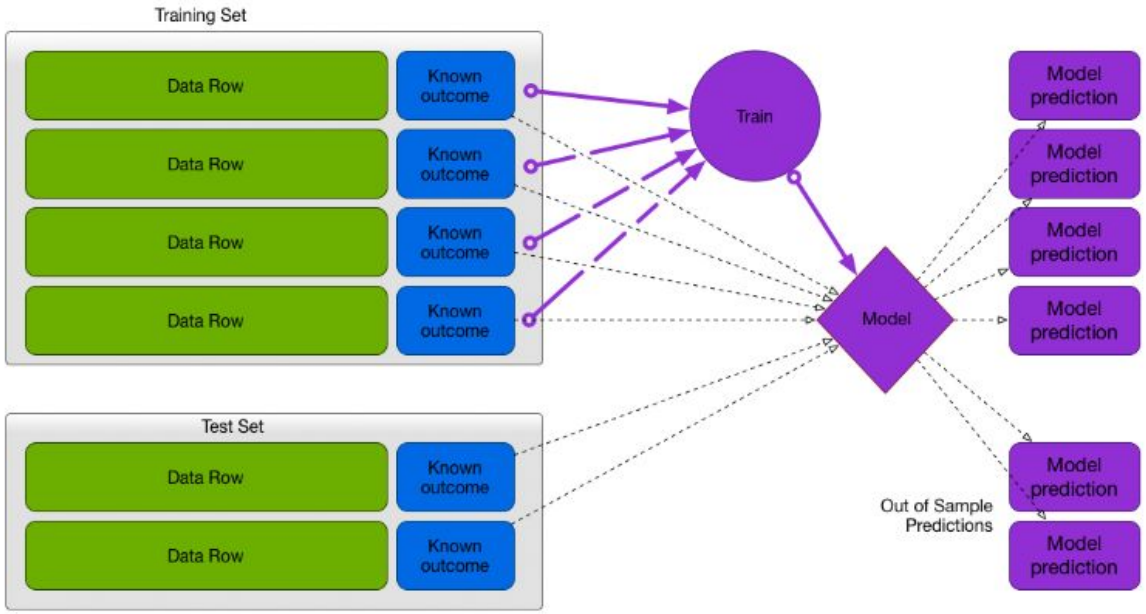
Root of Mean Squared Error

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2}$$

visit www.visual-design.net for step by step guide

Training & Fitting Machine Learning Models

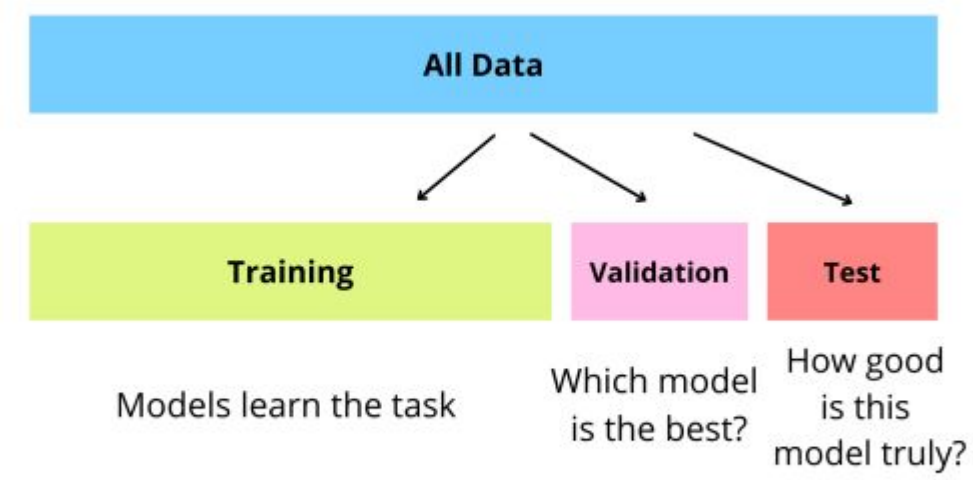
Train/Test Split



Recommended method when data is plentiful

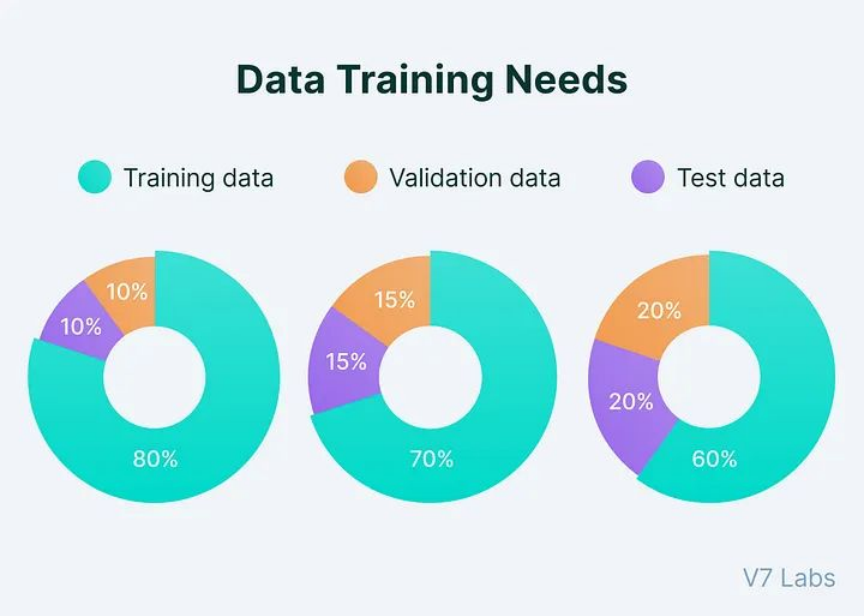
- A train/test split checks whether a model has learned general patterns or just memorized the training data.
- Ensures we evaluate a model fairly, using data that the model has not previously seen.

Three-Way Data Splitting



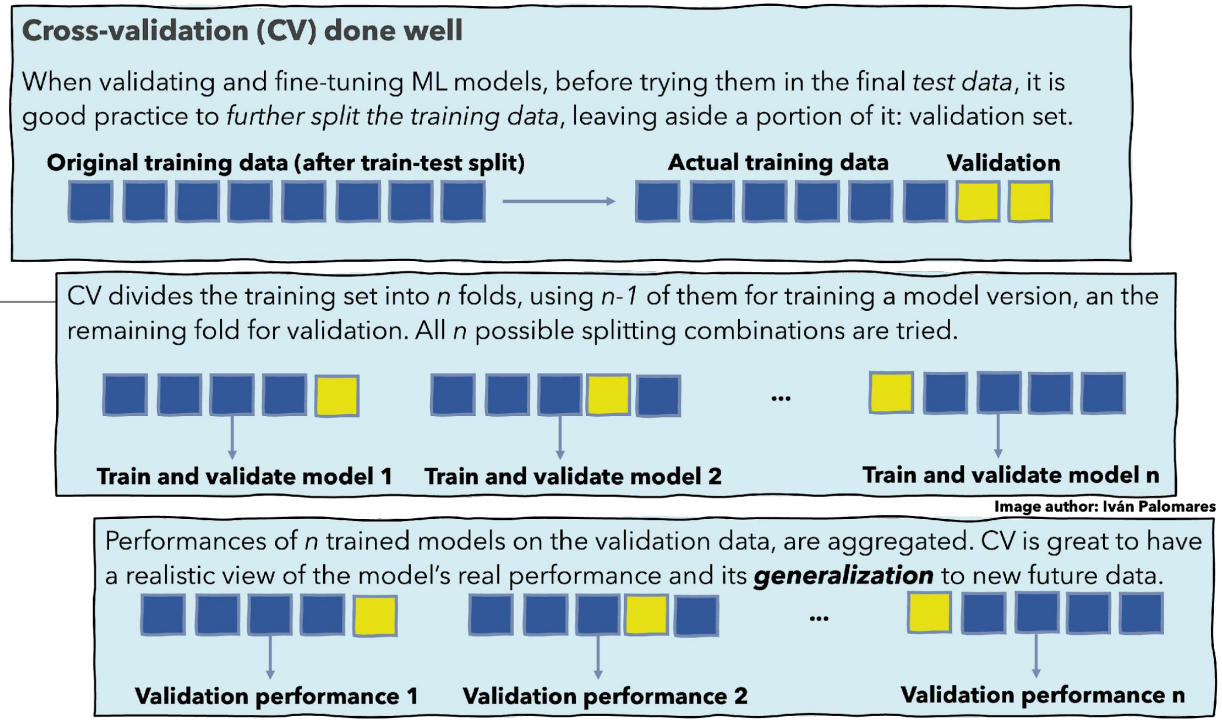
- **Training set**: data used to learn task/patterns
- **Validation set**: data used to refine model features/parameters (cross-validation)
- **Testing set**: data used to test performance

- *Essential in machine learning to evaluate generalization and detect overfitting/underfitting*



Images source:
https://medium.com/@rahulchavan4894/understanding-train-test-and-validation-dataset-split-in-simple-quick-terms-5a8630fe58c8

Cross-validation



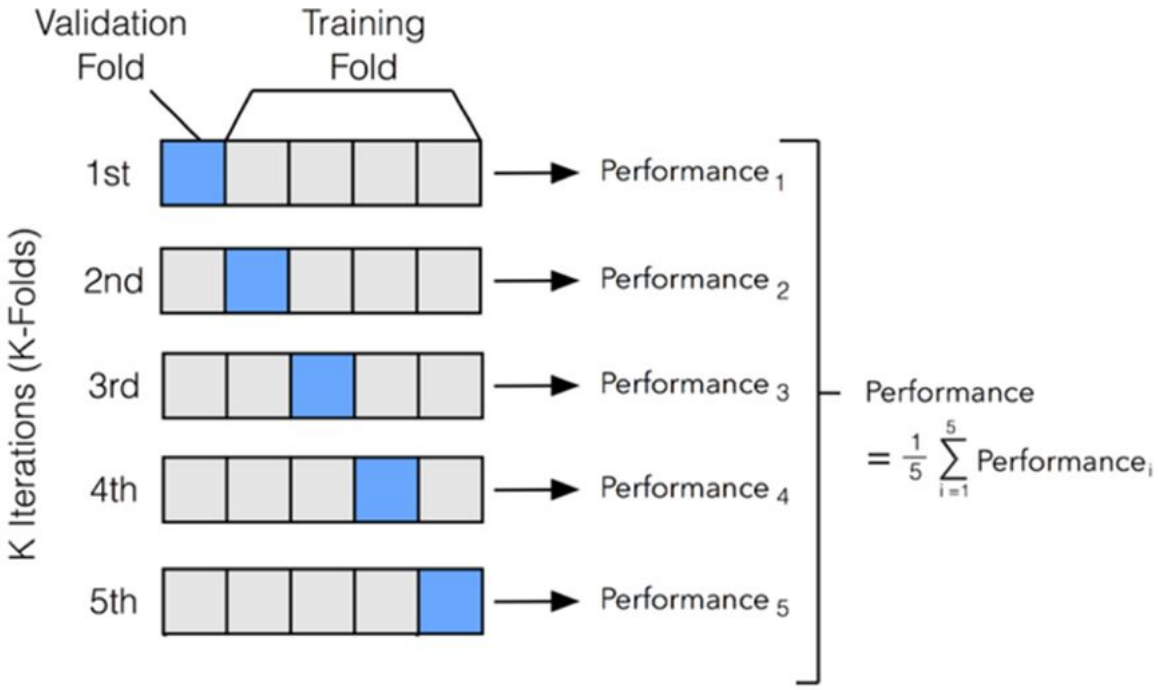
- Tool for estimating how well a model will generalize to new data
- Use on training set to:
  - Compare model features
  - Tune hyperparameters
  - Estimate performance
- Helps you choose models and settings without "cheating" by looking at test data

<u>Example</u>: *For our model to predict exam scores, we want to choose whether a linear regression or a polynomial regression is the best model. Use cross-validation and pick the model with the lowest average error*

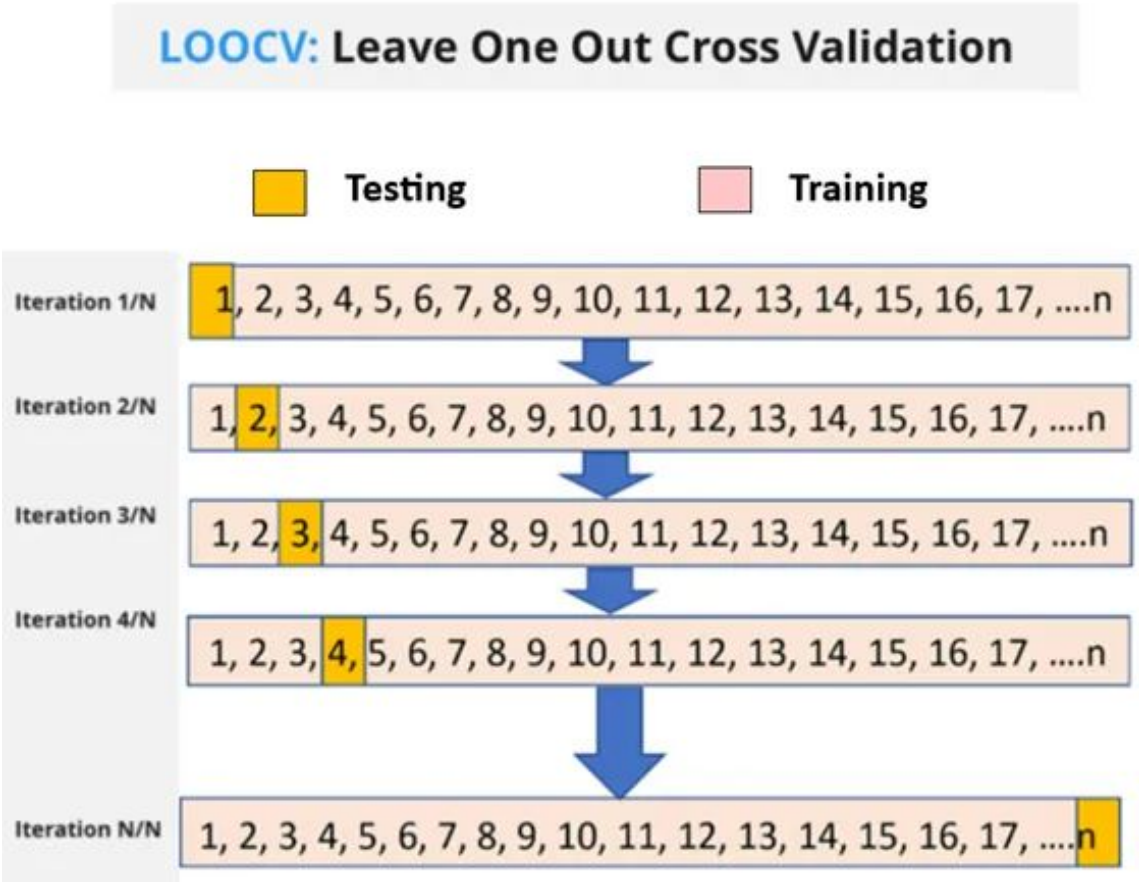
<u>Summary in context</u>: **Train/Test Split:** "Do I perform well on new data?"
Cross-Validation: "Which model should I choose before I test it?"

K-fold Cross-Validation

- Split data into K subsets or "folds" of equal size
- Train on K-1 folds and test on the remaining fold
- Repeat K times, each fold being used once as test
- Average performance across K iterations



LOOCV: Leave One Out Cross Validation



- Select n samples of the dataset
- Use n-1 samples of the dataset as the training set and 1 sample as the test set
- Repeat n times, each time leaving out a different sample as the test set
- Known as an exhaustive technique
- Pro: high accuracy (uses most of the samples of training set)
- Con: computationally expensive for large data

Model Evaluation

Why Evaluate Model Performance?

- Is the model accurate enough to make reliable predictions?
- Does it generalize well to unseen data?
- How does it compare to other models or benchmarks?
- What are the limitations and weaknesses of the model?

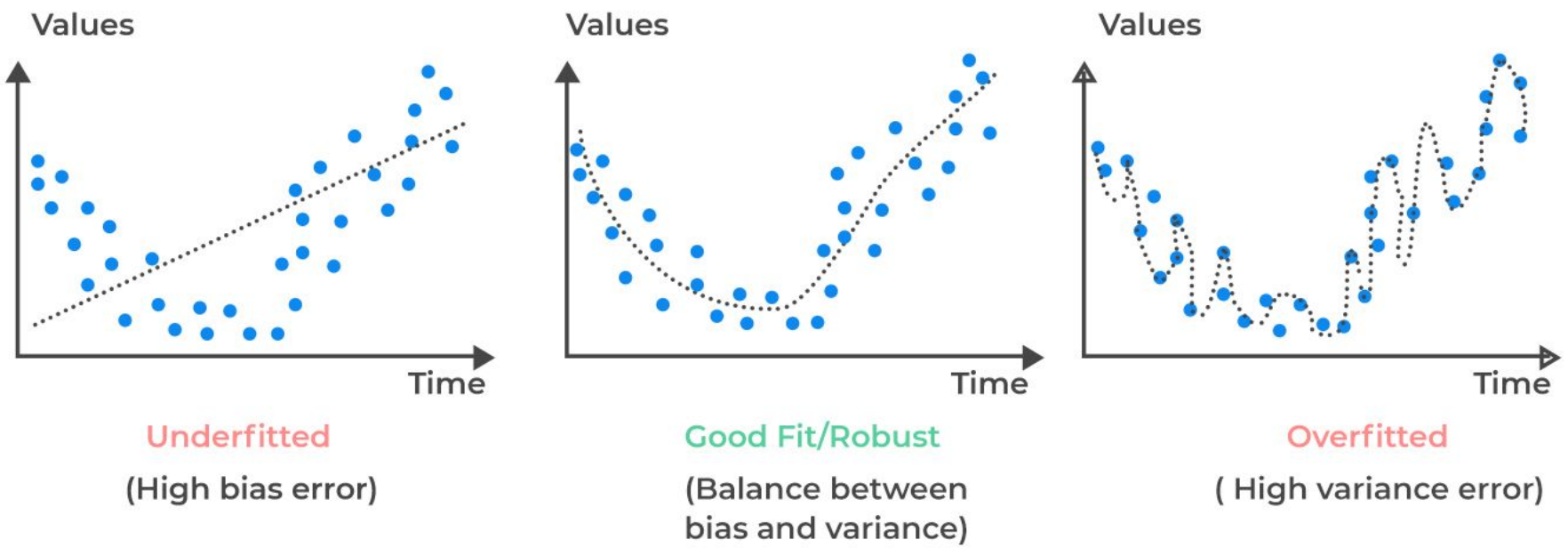
"Essentially, all models are wrong, but some are useful"

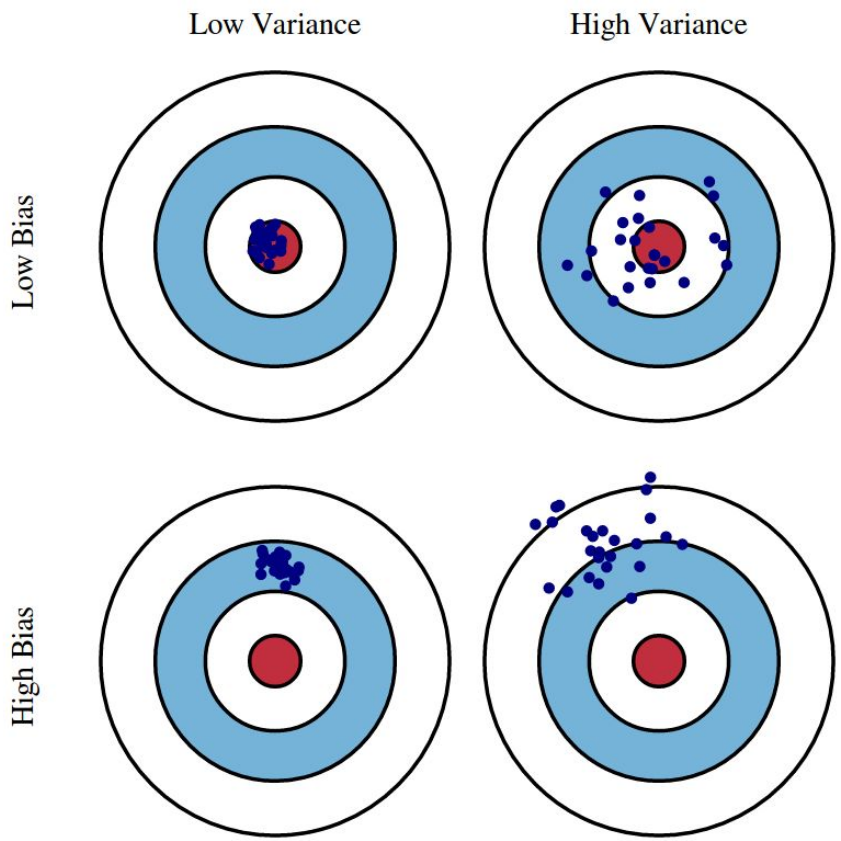
George E.P. Box

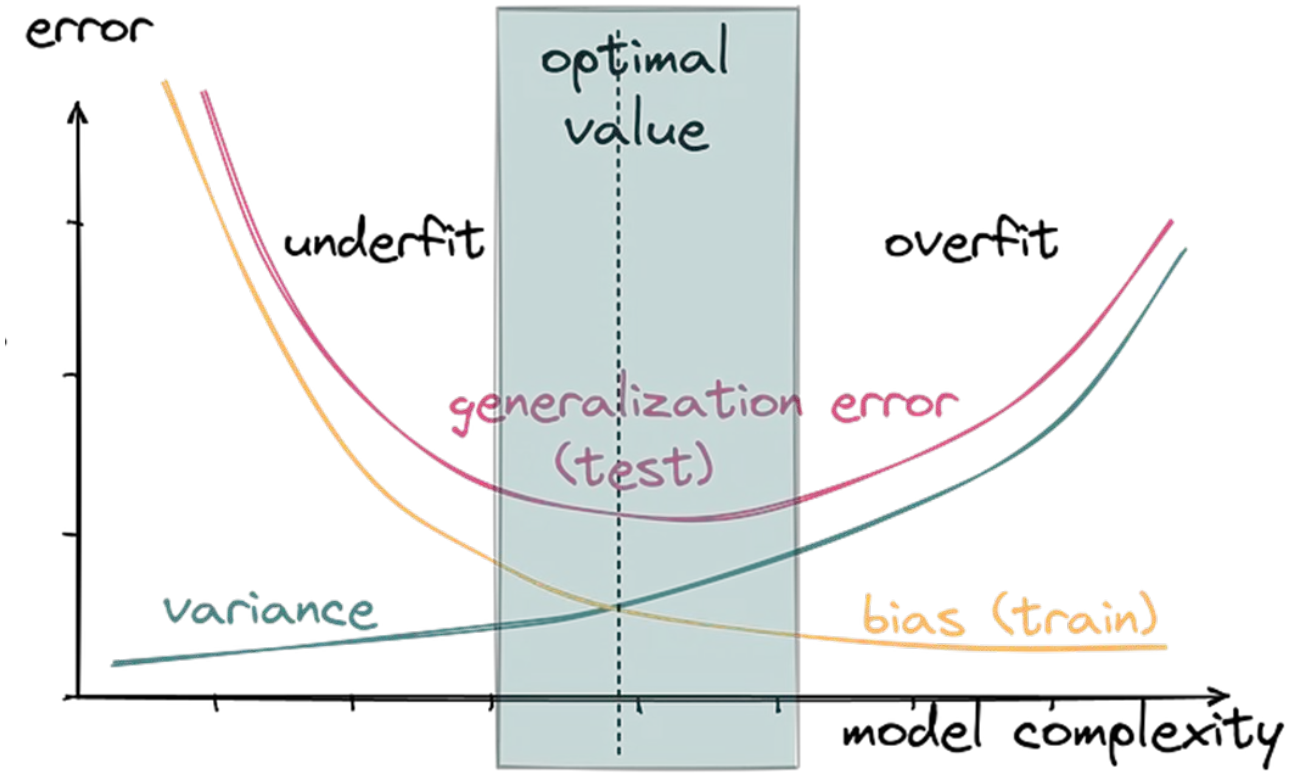
Common Metrics for Evaluation

- MSE (Mean Squared Error)
  - Advantages: differentiable, so easy to use as loss function
  - Disadvantages: squared unit output (interpretation); not robust to outliers
- RMSE (Root Mean Squared Error)
  - Advantages: same unit as output
  - Not robust to outliers
- MAE (Mean Absolute Error)
  - Advantages: Same unit as output variable; most robust to outliers
  - Disadvantages: Not differentiable (for optimization), so need optimizers like Gradient descent

Overfitting and underfitting describe how well a model balances learning patterns vs. generalizing to new data.







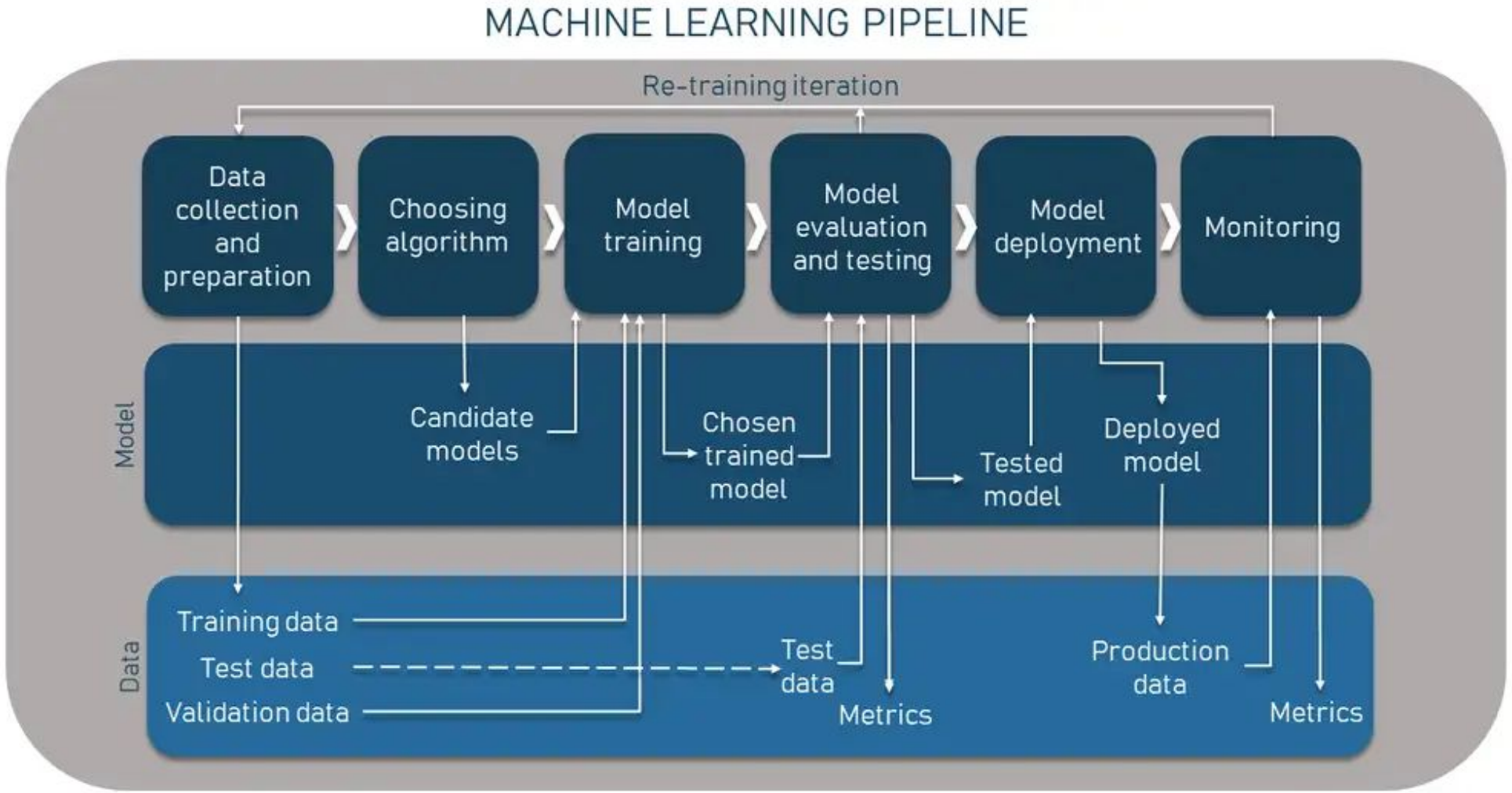
More complex models overfit while the simplest models underfit.

OVERFITTING VS UNDERFITTING IN MACHINE LEARNING

ASPECT	OVERFITTING	UNDERFITTING
Definition	The model performs well on training data but poorly on test data	The model performs poorly on both training and test data
Error Type	High variance, low bias	High bias, low variance
Model Complexity	Too complex	Too simple
Training Data	Learns noise and details in training data	Fails to capture underlying patterns in training data
Performance	Poor generalization to new data	Poor performance on both seen and unseen data
Symptoms	Low training error, high testing error	High training and testing error
Solution Approaches	Simplify the model, use regularization, add dropout	Increase model complexity, add more features, reduce regularization

@aiinsights.trends

A More Detailed/Formal Pipeline



altexsoft

Demo and In-Class Exercise: Week 8A